Interfaces: Default-Methoden

TL;DR
Seit Java8 können Methoden in Interfaces auch fertig implementiert sein: Sogenannte **Default-Methoden**.
Dazu werden die Methoden mit dem neuen Schlüsselwort `default` gekennzeichnet. Die Implementierung wird an die das Interface implementierenden Klassen (oder Interfaces) vererbt und kann bei Bedarf überschrieben werden.
Da eine Klasse von einer anderen Klasse erben darf, aber mehrere Interfaces implementieren kann, könnte es zu einer Mehrfachvererbung einer Methode kommen: Eine Methode könnte beispielsweise in verschiedenen Interfaces als Default-Methode angeboten werden, und wenn eine Klasse diese Interfaces implementiert, steht eine Methode mit der selben Signatur auf einmal mehrfach zur Verfügung. Dies muss (u.U. manuell) aufgelöst werden.
Auflösung von Mehrfachvererbung:

- Regel 1: Klassen gewinnen
- Regel 2: Sub-Interfaces gewinnen
- Regel 3: Methode explizit auswählen

Aktuell ist der Unterschied zu abstrakten Klassen: Interfaces können **keinen Zustand** haben, d.h. keine Attribute/-Felder.

Videos

- VL Default-Methoden
- Demo Regel 1
- Demo Regel 2
- Demo Regel 3

Unterlagen

- PDF-Version

Problem: Etablierte API (Interfaces) erweitern

```
interface Klausur {
    void anmelden(Studi s);
    void abmelden(Studi s);
}
```

=> Nachträglich noch `void schreiben(Studi s);` ergänzen?

Wenn ein Interface nachträglich erweitert wird, müssen alle Kunden (also alle Klassen, die das Interface implementieren) auf die neuen Signaturen angepasst werden. Dies kann viel Aufwand verursachen und API-Änderungen damit unmöglich machen.

Default-Methoden: Interfaces mit Implementierung

Seit Java8 können Interfaces auch Methoden implementieren. Es gibt zwei Varianten: Default-Methoden und statische Methoden.

```java
interface Klausur {
    void anmelden(Studi s);
    void abmelden(Studi s);

    default void schreiben(Studi s) {
        ...     // Default-Implementierung
    }

    default void wuppie() {
        throw new java.lang.UnsupportedOperationException();
    }
}
```

Methoden können in Interfaces seit Java8 implementiert werden. Für Default-Methoden muss das Schlüsselwort `default` vor die Signatur gesetzt werden. Klassen, die das Interface implementieren, können diese Default-Implementierung erben oder selbst neu implementieren (überschreiben). Alternativ kann die Klasse eine Default-Methode neu *deklarieren* und wird damit zur abstrakten Klasse.

Dies ähnelt abstrakten Klassen. Allerdings kann in abstrakten Klassen neben dem Verhalten (implementierten Methoden) auch Zustand über die Attribute gespeichert werden.

Problem: Mehrfachvererbung

Drei Regeln zum Auflösen bei Konflikten:

1. **Klassen gewinnen**: Methoden aus Klasse oder Superklasse haben höhere Priorität als Default-Methoden
2. **Sub-Interfaces gewinnen**: Methode aus am meisten spezialisiertem Interface mit Default-Methode wird gewählt Beispiel: Wenn `B extends A` dann ist `B` spezialisierter als `A`
3. Sonst: Klasse muss **Methode explizit auswählen**: Methode überschreiben und gewünschte (geerbte) Variante aufrufen: `X.super.m(...)` (`X` ist das gewünschte Interface)

Auf den folgenden Folien wird dies anhand kleiner Beispiele verdeutlicht.

Auflösung Mehrfachvererbung: 1. Klassen gewinnen

```java
interface A {
    default String hello() { return "A"; }
}
class C {
    public String hello() { return "C"; }
}
class E extends C implements A {}


/** Mehrfachvererbung: 1. Klassen gewinnen */
public class DefaultTest1 {
    public static void main(String... args) {
        String e = new E().hello();
    }
}
```

Demo: defaultmethods.rule1.DefaultTest1 (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/defaultmethods/rule1/DefaultTest1.java)

Die Klasse E erbt sowohl von Klasse C als auch vom Interface A die Methode `hello()` (Mehrfachvererbung). In diesem Fall "gewinnt" die Implementierung aus Klasse C.

1. Regel: Klassen gewinnen immer. Deklarationen einer Methode in einer Klasse oder einer Oberklasse haben Vorrang von allen Default-Methoden.

Auflösung Mehrfachvererbung: 2. Sub-Interfaces gewinnen

```
interface A {
    default String hello() { return "A"; }
}
interface B extends A {
    @Override default String hello() { return "B"; }
}
class D implements A, B {}


/** Mehrfachvererbung: 2. Sub-Interfaces gewinnen */
public class DefaultTest2 {
    public static void main(String... args) {
        String e = new D().hello();
    }
}
```

Demo: defaultmethods.rule2.DefaultTest2 (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/defaultmethods/rule2/DefaultTest2.java)

Die Klasse D erbt sowohl vom Interface A als auch vom Interface B die Methode `hello()` (Mehrfachvererbung). In diesem Fall "gewinnt" die Implementierung aus Klasse B: Interface B ist spezialisierter als A.

2. Regel: Falls Regel 1 nicht zutrifft, gewinnt die Default-Methode, die am meisten spezialisiert ist.

Auflösung Mehrfachvererbung: 3. Methode explizit auswählen

```
interface A {
    default String hello() { return "A"; }
}
interface B {
    default String hello() { return "B"; }
}
class D implements A, B {
    @Override public String hello() { return A.super.hello(); }
}


/** Mehrfachvererbung: 3. Methode explizit auswählen */
public class DefaultTest3 {
    public static void main(String... args) {
        String e = new D().hello();
```

```
    }
}
```

Demo: defaultmethods.rule3.DefaultTest3 (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/defaultmethods/rule3/DefaultTest3.java)

Die Klasse D erbt sowohl vom Interface A als auch vom Interface B die Methode `hello()` (Mehrfachvererbung). In diesem Fall *muss* zur Auflösung die Methode in D neu implementiert werden und die gewünschte geerbte Methode explizit aufgerufen werden. (Wenn dies unterlassen wird, führt das selbst bei Nicht-Nutzung der Methode `hello()` zu einem Compiler-Fehler!)

Achtung: Der Aufruf der Default-Methode aus Interface A erfolgt mit `A.super.hello();` (nicht einfach durch `A.hello();`)!

3. Regel: Falls weder Regel 1 noch 2 zutreffen bzw. die Auflösung noch uneindeutig ist, muss man manuell durch die explizite Angabe der gewünschten Methode auflösen.

Quiz: Was kommt hier raus?

```java
interface A {
    default String hello() { return "A"; }
}
interface B extends A {
    @Override default String hello() { return "B"; }
}
class C implements B {
    @Override public String hello() { return "C"; }
}
class D extends C implements A, B {}


/** Quiz Mehrfachvererbung */
public class DefaultTest {
    public static void main(String... args) {
        String e = new D().hello(); // ???
    }
}
```

Die Klasse D erbt sowohl von Klasse C als auch von den Interfaces A und B die Methode `hello()` (Mehrfachvererbung). In diesem Fall "gewinnt" die Implementierung aus Klasse C: Klassen gewinnen immer (Regel 1).

Beispiel: defaultmethods.quiz.DefaultTest (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/defaultmethods/quiz/DefaultTest.java)

Statische Methoden in Interfaces

```java
public interface Collection<E> extends Iterable<E> {
    boolean add(E e);
    ...
}
```

```
5  public class Collections {
6      private Collections() { }
7      public static <T> boolean addAll(Collection<? super T> c, T... elements)
           {...}
8      ...
9  }
```

Typisches Pattern in Java: Interface plus Utility-Klasse (Companion-Klasse) mit statischen Hilfsmethoden zum einfacheren Umgang mit Instanzen des Interfaces (mit Objekten, deren Klasse das Interface implementiert). Beispiel: `Collections` ist eine Hilfs-Klasse zum Umgang mit `Collection`-Objekten.

Seit Java8 können in Interfaces neben Default-Methoden auch statische Methoden implementiert werden.

Die Hilfsmethoden können jetzt ins Interface wandern => Utility-Klassen werden obsolet … Aus Kompatibilitätsgründen würde man die bisherige Companion-Klasse weiterhin anbieten, wobei die Implementierungen auf die statischen Methoden im Interface verweisen (*SKIZZE, nicht real!*):

```
1  public interface CollectionX<E> extends Iterable<E> {
2      boolean add(E e);
3      static <T> boolean addAll(CollectionX<? super T> c, T... elements) { ... }
4      ...
5  }
6  public class CollectionsX {
7      public static <T> boolean addAll(CollectionX<? super T> c, T... elements) {
8          return CollectionX.addAll(c, elements);  // Verweis auf Interface
9      }
10     ...
11 }
```

Interfaces vs. Abstrakte Klassen

- **Abstrakte Klassen**: Schnittstelle und Verhalten und Zustand
- **Interfaces**:
  - vor Java 8 nur Schnittstelle
  - ab Java 8 Schnittstelle und Verhalten

  Unterschied zu abstrakten Klassen: Kein Zustand, d.h. keine Attribute

- Design:
  - Interfaces sind beinahe wie abstrakte Klassen, nur ohne Zustand
  - Klassen können nur von **einer** (abstrakten) Klasse erben, aber **viele** Interfaces implementieren

Wrap-Up

Seit Java8: Interfaces mit Implementierung: **Default-Methoden**

- Methoden mit dem Schlüsselwort `default` können Implementierung im Interface haben
- Die Implementierung wird vererbt und kann bei Bedarf überschrieben werden
- Auflösung von Mehrfachvererbung:
  - Regel 1: Klassen gewinnen
  - Regel 2: Sub-Interfaces gewinnen
  - Regel 3: Methode explizit auswählen
- Unterschied zu abstrakten Klassen: **Kein Zustand**

Zum Nachlesen

- Oracle Corporation (2024)
- Urma, Fusco, und Mycroft (2014, Kap. 9)

Lernziele

- k2: Interfaces mit Default-Methoden, Unterschied zu abstrakten Klassen
- k2: Problem der Mehrfachvererbung
- k3: Erstellen von Interfaces mit Default-Methoden
- k3: Regeln zum Auflösen der Mehrfachvererbung

Challenges

Erklären Sie die Code-Schnipsel in der Vorgabe und die jeweils entstehenden Ausgaben.

Quellen

Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.

Urma, R.-G., M. Fusco, und A. Mycroft. 2014. *Java 8 in Action: Lambdas, Streams, and Functional-Style Programming*. Manning Publications.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)